Best Practices for Programming with AI Agents: Getting Started with Task Delegation

Jerod W. Wilkerson

October 2026

AI attribution: drafted with Claude (Anthropic) under the author's direction, and reviewed and edited by the author.

1. Who This Guide Is For

This guide is for BYU computer science students who use AI coding agents on substantial course assignments where AI use is permitted. You do not need prior experience with AI tools. You do need to be comfortable with Git, a terminal, and at least one programming language.

The guide teaches one way of working. You still write code yourself, and you also delegate bounded tasks to an agent: implement this function, fix this bug, write tests for this module. You review everything the agent produces, integrate the results yourself, and remain responsible for what ends up in your submission.

Students most often start in one of two other ways. One is accepting autocomplete suggestions, which leaves you writing nearly all of the code. The other is prompting an agent until the program appears to work without reading the code it wrote, which can produce something that runs but that you cannot explain, debug, or trust.

The AI Fluency Ladder

A useful way to place this way of working is the AI Fluency Ladder. The framework was developed by Dan Shapiro, modeled on the levels of driving automation, and popularized by Nate B. Jones. The version used here is my adaptation. Each rung describes how much of the development process you can reliably delegate to AI, and Table 1 summarizes the levels of fluency.

Level	Name	What You Do
0	Vibe Coding	Describe what you want and accept whatever seems to work
1	Smart Autocomplete	Write code and accept or reject inline suggestions
2	Task Delegation	Write code, delegate bounded tasks, review and integrate the results
3	Agentic Workflows	Design workflows of agents and review their implementations
4	Verified Execution	Define features and verify outcomes instead of reviewing code
5	Autonomous Execution	Define objectives and evaluate the finished software

Table 1. The AI Fluency Ladder

Vibe Coding sits below the ladder because the person doing it cannot judge the code they receive. This guide teaches the Task Delegation level of fluency, where an agent first completes whole pieces of work for you. Most professional software developers work at this level today, so getting started here does not mean working at a beginner's level, and every level above it depends on the habits this guide teaches.

How This Fits the BYU CS AI-Use Policy

The BYU Computer Science department gives every assignment an AI-use level from 0 to 4: Level 0 (No AI), Level 1 (Reference), Level 2 (Reviewer), Level 3 (Partner), and Level 4 (Full Use). These numbers are unrelated to the levels of fluency in Table 1, so this guide always names the scale it means. At AI-use Levels 0 through 2, an agent may not write any of your work, so this guide is written for assignments at AI-use Level 3 (Partner) and AI-use Level 4 (Full Use). Section 3 covers the policy in more detail, and the full text is at cs.byu.edu/department/ai-policy.

What This Guide Covers

The guide starts with how AI coding agents behave and how they fail, then covers academic integrity and learning, choosing and setting up a tool, defining a task, the working loop for each task, reviewing the agent's work, debugging and knowing when to start over, and cost. It closes with the signs that you have outgrown this way of working and what comes next.

2. How These Tools Behave and How They Fail

Suppose you ask an agent to write a login function for a small web application. The code it returns is clean, passes the linter, handles an empty username and a missing password field, and comes with a tidy explanation. When you read it more carefully, you notice that it saves each password to the database exactly as the user typed it, with no hashing. The code compiles, the tests the agent wrote pass, and the login works. That gap between looking right and being right is the reason this guide spends so much time on review.

Fluent Output Is Not Evidence of Correct Output

Most of the tools you use as a programmer are deterministic: give a compiler the same source file twice and you get the same result twice. The model behind an AI coding agent generates text one small piece at a time, choosing at each step among plausible continuations according to probabilities learned from its training text. Better prompts narrow those choices but do not remove the variation. Ask for the same function twice and you may get two different implementations, or one correct and one subtly wrong.

The model also produces correct and incorrect answers the same way. A fabricated method name and a real one arrive with the same formatting and the same confident tone. So does the summary an agent writes about its own work, which is generated text like everything else and can describe work that was never done. Confidence is a poor indicator of correctness, and a detailed explanation is not evidence that the code works.

What the Agent Can See

An agent works only from what is in its context window: the files it has read, the commands it has run and their output, your instructions, and the conversation so far. It has no memory of yesterday's session and no view of your assignment specification or the parts of the codebase it never opened, unless that material is put in front of it. When an agent ignores a requirement, first ask whether the requirement was ever in its context.

The context window is also finite. Every file read, test run, and failed attempt takes up space, and as a session grows, the quality of the agent's work often declines. Instructions you gave at the start, such as "don't add new dependencies," can lose their effect even though they are still in the conversation, so the agent may follow a rule for the first several changes and break it an hour later. This is why later sections recommend small tasks and a fresh session for each one.

Failure Modes You Will Run Into

AI-generated code tends to fail in recognizable ways, and knowing them tells you where to look when you review.

Hallucinated APIs and imports. The agent calls a function that does not exist, passes an argument a function does not accept, or imports a package that was never published, usually in the library's own naming style. For example, an agent might produce this call:

```
const response = await fetch(url, { method: "GET", timeout: 5000 });
```

The options object that `fetch` accepts has no `timeout` property, and the TypeScript compiler usually rejects a line like this. Other invented names get through: code typed as `any`, plain JavaScript files, libraries without type definitions, and names inside strings, such as route paths, configuration keys, and environment variable names. A request to `/api/user/profile` when the server defines `/api/users/profile` compiles cleanly and fails only when it runs.

Plausible-but-wrong logic. The code runs and produces reasonable-looking output, but it does not do what the specification says. Off-by-one errors, reversed comparisons, and sorting in the wrong direction are typical. Suppose your assignment asks for a function that splits a list into pages of `size` items and returns the requested page, with pages numbered starting at 1. The agent writes this:

```
function getPage<T>(items: T[], page: number, size: number): T[] {
  const start = page * size;
  return items.slice(start, start + size);
}
```

The formula `page * size` assumes pages are numbered from 0. With pages numbered from 1, page 1 should start at index 0, but this code starts it at index `size`. For a list of six items in pages of two, requesting page 1 returns the third and fourth items, which are page 2. Requesting page 3 returns an empty array, and no valid page number returns the first two items. The correct formula is `(page - 1) * size`.

Outdated library usage. A model learns from code written over many years, so it often reaches for a deprecated API or an outdated default, such as `new Buffer(data)`, a Node.js constructor deprecated for years in favor of `Buffer.from(data)`. Deprecation warnings are often the first sign.

Partial completion reported as complete. You ask for a report feature with a CSV export, and the agent adds the database query, the endpoint, the export button, and a summary that says the feature is finished. Inside the export function you find this:

```
export function exportCsv(report: Report): string {
  // TODO: implement CSV export
  return "";
}
```

The large amount of finished work makes the unfinished piece easy to miss.

Silent failures. Something required is skipped, and nothing reports an error. You ask for a login handler that locks an account after five failed attempts, and the agent implements everything except the lockout. The code compiles and every test passes, because no test checks for the lockout. You find silent failures only by checking each requirement against the code.

Sycophancy. Models are trained in part on human approval, so they tend to shift toward what you appear to believe. Ask "Shouldn't this loop condition be `<=` instead of `<`?" and an agent may agree, apologize, and introduce an off-by-one error into code that was correct. Its agreement looks the same whether or not you are right, so it is not evidence that you are.

Tests that confirm the agent's own mistakes. When the agent writes both the code and its tests, the tests usually encode the same assumptions as the code. If the agent misread the page numbering in the `getPage` example, its test will likely include this assertion:

```
expect(getPage(["a", "b", "c", "d"], 1, 2)).toEqual(["c", "d"]);
```

The test passes and confirms the bug. A passing suite written by the session that wrote the code shows only that the code matches the agent's interpretation of the task.

Check Mechanically What You Can

Use deterministic checks for everything that can be checked mechanically. The TypeScript compiler, `tsc`, catches many invented names, wrong signatures, and mismatched types before the program runs, though not the string-based and untyped cases above. A linter can flag deprecated calls and swallowed exceptions. Tests that you wrote or specified yourself catch logic that does not match the assignment. These tools give the same answer every time, however confident the agent sounded. Save your own judgment for what no tool can check: whether the code does what the assignment asked, whether a requirement is missing, whether a design choice makes sense, and whether you could explain the code to your instructor.

3. Academic Integrity and Learning

BYU's Academic Honesty Policy

BYU's Academic Honesty Policy does not mention AI, but two of its definitions apply directly to agent-written code. Plagiarism is "representing the words, ideas, or data of another as one's own" without proper attribution. The policy was written before AI agents, so it does not say whether an agent counts as "another," but if your instructor requires you to disclose AI use and you do not, submitting the agent's code as your own can be treated as plagiarism. Cheating is an attempt to "give the appearance of a level of knowledge or skill that the student has not obtained." Submitting code you could not have written and cannot explain fits that one, even when you disclosed every line.

BYU has no campus-wide rule for AI use on coursework. Each instructor sets the rules for their own courses, and BYU's guidance for students says that "the professor is the arbiter of academic integrity." If neither the syllabus nor the assignment addresses AI, the same guidance says to "assume that using GenAI is not permitted." If you are not sure, ask before you start.

The CS Department's AI-Use Levels

Every assignment in a CS course has exactly one AI-use level. Your syllabus sets the default for the course, and an individual assignment can change it, so check both.

The levels are defined by two questions: can AI see your work, and can AI write it? Seeing includes pasting code into a chat, uploading a screenshot, or working in an AI-enabled editor with your files available to the agent. Writing means AI-produced content ends up in what you submit, whether you paste it or retype it. Table 2 summarizes the five levels by those two questions.

Level	Name	AI Sees Your Work	AI Writes Your Work
0	No AI	No	No
1	Reference	No	No
2	Reviewer	Yes	No
3	Partner	Yes	Yes, with you
4	Full Use	Yes	Yes, most or all of it

Table 2. CS Department AI-Use Levels

Some rules apply at every level. You are responsible for everything you submit and must be able to explain and justify it. Your instructor may ask you to explain your work out loud, modify or extend it live, or complete a timed follow-up without AI, and not being able to explain or adapt your submission may be treated as evidence that you went past the assigned level.

At AI-use Level 3 (Partner), the agent writes code that you review, and you design and debug alongside it. At AI-use Level 4 (Full Use), the agent may generate most or all of the code and tests, and your job shifts toward specifying the work and verifying the result. The policy's list of what not to do at Full Use includes skipping the specification and claiming the code works without evidence that shows it does. There, the task definitions in Section 5 become your specification, and the checks and review in Sections 6 and 7 become your evidence that the code meets it.

Attribution

Follow whatever disclosure your instructor requires. When an instructor allows AI use but does not say how to disclose it, a sensible default has two parts. The first is an attribution log, a file named `AI-ATTRIBUTION.md` in the root of your repository, with one entry per task. A typical entry looks like this:

```
## 2026-10-14: src/routes/scores.ts (new file)
- Tool: <tool name>
- Task: Add GET /api/courses/:id/scores, returning scores sorted
```

```
  by submission date, with a 404 for an unknown course.
- Review: Changed the 404 response to match the existing error
  format. Wrote the sorting and unknown-course tests myself.
```

The second is a one-line marker at the top of each file that contains agent-written code, so anyone who opens the file knows to check the log:

```
// AI-assisted: see AI-ATTRIBUTION.md
```

You can have the agent draft these entries through an instruction in your project instruction file, and Appendix A gives the wording. Check every entry, though. An entry is the agent's summary of its own work (Section 2), so it can list a change that was never made or leave out one that was. An inaccurate attribution log is a disclosure problem of its own, and it is still yours.

Learning Acceleration and Learning Bypass

You can disclose every line an agent wrote and still finish a course without the skills it was meant to teach. Using an agent to accelerate learning looks like this: you attempt the problem, get stuck, ask the agent to explain the concept you are missing, read every line it writes, and write the next similar piece yourself. Using it to bypass learning looks like pasting the assignment into the agent and submitting what comes back. That can produce working code, but the grade no longer reflects what you can do.

A short self-test tells you which side you are on. Could you explain this code line by line to a classmate? Could you write something equivalent without the tool? Could you debug it if it broke? If the answer to any of these is no, fill the gap before you move on.

The self-test is also the prerequisite for Task Delegation, because you cannot review code you do not understand. The plaintext-password bug in Section 2 was visible only to someone who knew that passwords should be hashed, and the other failure modes in that section are caught the same way.

Professional Considerations

Three concerns carry over into professional work. The first is intellectual property. Models are trained on published code under many different licenses, and whether their output can be treated as a derivative of that code is legally unsettled, so in open-source or commercial work, follow your employer's or project's rules.

The second is confidentiality. When an agent reads a file, the contents typically go to the tool provider's servers, and an agent in your project folder can read whatever is there, including `.env` files with API keys, credentials, and data files, so be deliberate about what is in the folder. For university data, BYU's student guidance points to its Data Use, Privacy, and Security Policy, and employers usually have similar rules.

The third is accountability. Whoever commits the code owns it, including any bug or security hole the agent introduced.

4. Choosing and Setting Up a Tool

This guide covers four AI coding agents, GitHub Copilot, Cursor, Claude Code, and OpenAI Codex, and endorses all four equally. Each can do everything this guide teaches, so pick the one you find comfortable. Claude Code and Cursor are among the most heavily used AI coding tools in industry, according to the Pragmatic Engineer's survey of developers in early 2026, so time you spend learning either one carries into professional work.

Working in an Editor

Most students are more comfortable in an editor than in a terminal, and all four tools work inside a VS Code-style editor. Cursor is itself an editor, built on VS Code. Copilot is built into VS Code and is also available in JetBrains IDEs. Claude Code and Codex each have a VS Code extension as well as a terminal version, so you can use any of the four without leaving your editor.

The Four Tools

Plans and prices below are as of October 2026 and change often.

GitHub Copilot. Copilot is GitHub's AI coding assistant. Verified students can get the free Copilot Student plan through GitHub's education benefits at github.com/settings/education/benefits. The plan includes agent mode and uses automatic model selection only. GitHub paused new sign-ups for the plan in April 2026 and began reopening them gradually in June, so check the benefits page for current status. Activation can take several days after GitHub verifies you as a student. This guide uses agent mode in the Chat view, where you give the agent a whole task, not Copilot's inline completions.

Cursor. Cursor is an editor with an agent built in. It has no student plan at present, and its free tier includes a limited number of agent requests. The Plus plan ($8/month) includes the agent but offers only Grok models. Claude and GPT models require the Premium plan ($30/month), with use of those models billed separately at API prices.

Claude Code. Claude Code is Anthropic's coding agent. It runs in a terminal or through its VS Code extension. It requires a paid Claude plan, and the cheapest is Pro at $20/month. Claude's free plan does not include Claude Code, and Anthropic offers no individual student discount.

OpenAI Codex. Codex is OpenAI's coding agent, and it also runs in a terminal or through an editor extension. Students should have free access through BYU, which is rolling out ChatGPT Edu to students now. Check ai.byu.edu for current status. Separately, OpenAI offers verified university students in the United States and Canada $100 in credits for use in Codex, one offer per student. The credits expire 12 months after they are granted.

Comparing the Tools

Later sections describe practices in tool-neutral terms, and Table 3 shows where each one lives in each tool.

	GitHub Copilot	Cursor	Claude Code	OpenAI Codex
Where it runs	VS Code; JetBrains IDEs	Cursor's own editor	VS Code extension; terminal	VS Code extension; terminal
Project instruction file	`AGENTS.md` (read by default)	`AGENTS.md`	`CLAUDE.md`, or `AGENTS.md` if there is no `CLAUDE.md`	`AGENTS.md`
Permission modes	Manual permissions (default); Allow all and Autopilot skip approval	Allowlist or Auto-review (default); Run Everything skips approval	Manual (default); Bypass permissions skips approval	Ask for approval; Full access skips approval
How to review and undo changes	Keep or Undo per file or change; Restore Checkpoint	Diff view, reject what you don't want; Restore Checkpoint	Side-by-side diff to accept or reject; `/rewind` restores a checkpoint	Keep only the changes you want; commit in Git before and after each task
How to start a fresh session	Cmd/Ctrl+N or `/clear` in the Chat view	Cmd/Ctrl+N	`/clear`, or Open in New Tab in VS Code	`/new` in the terminal version
Built-in worktree support	Yes, worktree sessions	Yes, `/worktree`	Yes, `claude --worktree`	Yes, `/worktree`
Student access (October 2026)	Free Copilot Student plan	No student plan; free tier with limited agent requests	Paid plan required; Pro is $20/month	Should be free through BYU (see ai.byu.edu); $100 student credit offer

Table 3. The Four Tools Compared

The Project Instruction File

Every one of these tools reads a project instruction file: a short Markdown file of standing instructions that the agent loads at the start of each session. Create a single file named `AGENTS.md` at the root of your repository and commit it to Git. Copilot in VS Code, Cursor, and Codex read `AGENTS.md` automatically, and Claude Code reads it when the project has no `CLAUDE.md`. If you also want a `CLAUDE.md`, put the line `@AGENTS.md` at its top so Claude Code loads both.

The file should tell the agent what the project is, how to build and check it, which conventions to follow, and what it must not touch. Suppose your assignment is a REST API for course scheduling, written in TypeScript on Node with Express. This guide uses that project as its running example. Its instruction file might look like this:

```
# AGENTS.md

## Project
REST API for a course-scheduling assignment. TypeScript on Node with
Express. Source is in src/, tests are in test/ and use Jest.

## Commands
- Build: npm run build
- Test: npm test
- Lint: npm run lint
- Type check: npx tsc --noEmit

## Conventions
- Use named exports, not default exports
- Do not use the `any` type
- Return errors in the existing { "error": "..." } JSON format

## Off-limits
- Do not edit files in src/generated/ (the build produces them)
- Do not change the starter tests in test/starter/ (provided by the instructor)
- Do not add npm dependencies without asking
```

Keep yours about this short. Every line takes up room in the context window in every session, and a long file of rarely relevant rules makes the important ones easier to lose. Add a line when the agent makes the same mistake twice, and remove lines that no longer apply.

What the Instruction File Cannot Guarantee

Do not rely on the instruction file for anything that must hold every time. The file is part of the agent's context, and instructions can lose their effect as a session grows (Section 2). A line that says "never delete files" usually works, and that is not good enough for a rule whose failure costs you your work.

For control over what the agent actually does, use the tool's permission mode (Table 3). Start in a mode where the agent asks before running commands, and loosen it only for actions you have watched it handle well. Do not use the modes that skip all approval (Allow all or Autopilot, Run Everything, Bypass permissions, or Full access) on your own machine. Even the stricter modes are a convenience. The VS Code documentation calls its terminal approval rules "a best-effort convenience, not a security boundary," and Cursor describes its own permission controls as best-effort as well.

Your real protection is Git and your own checks. If every task starts from a clean commit, any change the agent makes can be inspected and thrown away (Section 6).

5. Defining a Task

Everything else in this guide depends on turning a piece of your assignment into a task an agent can complete and you can review.

What Counts as a Task

A task is a bounded piece of implementation work with a clear objective, a defined scope, and a complete result you can review as a finished piece of work. In the course-scheduling API from Section 4, each of the following is a task: implement the endpoint that lists a student's enrolled sections, fix the bug that lets a section enroll more students than its capacity, refactor the duplicated validation code in the routes module into one shared function, or write tests for the existing waitlist logic. For each one, you can say what the agent should change, what it should leave alone, and how you will know it finished.

Some of the work stays with you: integration, the small corrections after review that are faster to write than to describe, and logic that needs close attention, such as the central algorithm an assignment is designed to teach.

A Vague Request and a Bounded One

Suppose your API needs to stop students from enrolling in two sections that meet at the same time. A first attempt at the request often looks like this:

```
Add schedule-conflict checking to enrollment so students can't sign up
for overlapping classes.
```

The same work, defined as a bounded task, looks like this:

```
Add a schedule-conflict check to POST /api/sections/:id/enrollments.

Objective: reject an enrollment when the student is already enrolled in
a section whose meeting times overlap the new section's. Return 409 with
{ "error": "Schedule conflict with <section code>" }.

Read first: src/routes/enrollments.ts, src/services/enrollment.ts, and
the MeetingTime type in src/models/section.ts.

Constraints:
- Put the overlap logic in src/services/enrollment.ts as a named export,
  not in the route handler
- Two meeting times overlap only if they share a day and their time
  ranges intersect (a class ending at 10:50 does not conflict with one
  starting at 10:50)
- Do not change the responses for successful enrollments or for the
  existing capacity check
- Do not modify src/models/section.ts or anything in test/starter/
- Do not add npm dependencies

Done when:
- New tests in test/enrollment.test.ts cover overlapping times, the same
  day without overlap, back-to-back sections, and different days
- npm test, npx tsc --noEmit, and npm run lint all pass

Some sections may have no meeting times (online sections, for example).
If you find any, ask me how to treat them instead of deciding.
```

The first request leaves the agent to guess where the check belongs, which status code and error message to return, whether two classes that touch at 10:50 conflict, whether the model or the existing capacity check

should change, and whether tests are part of the job. An agent will make every one of those decisions, usually without saying which, and each guess that is reasonable but wrong for your assignment, such as treating back-to-back classes as a conflict, becomes work for you in review. The boundary case is the hardest to catch, because the agent's own tests will likely agree with its guess (Section 2). The bounded version settles these questions before any code exists, names the files to read, and gives you a concrete list to check the result against.

What a Task Definition Includes

The bounded request above has five parts, and most good task definitions have the same ones:

- The objective: what the change should accomplish, stated specifically enough that you could tell whether it did
- The files or code to read first, so the agent starts from the right part of the project
- Constraints and what must stay unchanged, including interfaces, response formats, and files that are off limits
- Done criteria and the checks that must pass, such as specific tests, the type check, and the linter
- Anything the agent should ask about instead of deciding, usually a choice the assignment or your instructor would make

A small bug fix may need only one sentence of objective and one line of checks, and standing rules already in your project instruction file can be left out.

When you are not sure how to approach a task, ask the agent for a plan before it edits anything: the approach it intends to take and the files it expects to change, with an instruction not to modify anything yet. A plan that misreads the assignment, touches files it should not, or skips a requirement is much cheaper to correct in a paragraph than in a finished change.

Sizing a Task

The size of a task can go wrong in two directions. Over-scoping asks for too much at once. "Implement enrollment, waitlists, and schedule conflicts" typically produces a result that almost works: most of the endpoints respond, most of the tests pass, and the remaining problems are spread across many files. Reviewing a change that large is slow, and repairing it often takes longer than writing a smaller version would have.

Under-scoping breaks the work into pieces so small that each one is done without enough context to fit with the others. One task adds a helper that parses meeting times, another adds a near-duplicate because the first was not in its context, and a third writes a route that expects a different format from both. You end up as the integrator, reconciling code that was never designed together.

To find a size between those two, use this rule of thumb when you define a task: it should be small enough that you can read the whole change carefully in one sitting and explain every line, and large enough to produce a complete result that you could commit by itself. The schedule-conflict task meets both conditions. The combined enrollment request is too large to read in one sitting, and a task that only adds the overlap helper, with nothing that calls it, barely produces a result worth committing. When a task is too large to read in one sitting, split it. Smaller tasks also help the agent, because each one can start in a fresh session with your instructions near the beginning of a nearly empty context window (Section 2). Appendix A gives templates for the tasks you will define most often.

6. The Working Loop

Each task goes through the same routine, whether it is a one-line bug fix or the schedule-conflict check, and the routine depends mostly on Git. The steps for one task are these:

1. Start from a clean Git state
2. Start a fresh session and give the agent the task, in the permission mode you chose in Section 4
3. While the agent works, do something that does not change its files, or work in a separate worktree
4. When the agent reports that it is done, look at what changed

5. Run the build, tests, and linter yourself
6. Review the change (Section 7)
7. Decide whether to keep it, fix it, ask for a correction, or discard it
8. Commit with a message that notes agent involvement, and update your attribution log (or confirm that your agent did)
9. Start the next task in a fresh session

Start Clean

Before you start the agent, commit or stash your own work so that `git status` reports a clean working tree. Use `git stash -u` if you have new files you are not ready to commit, since a plain `git stash` leaves untracked files behind. Every change afterward is then the agent's, and you can throw all of it away without losing anything of yours.

Look at What Changed, Not at the Summary

When the agent finishes, it usually summarizes what it did. Because that summary is generated text (Section 2), look at the change itself:

```
git status
git diff
```

`git status` lists every modified file and, under "Untracked files," every new one. Read that list first, since an unexpected file, such as anything in `test/starter/`, tells you something before you read the diff. `git diff` does not show the contents of new files, so open those directly, or use your tool's diff view (Table 3). If your permission mode lets the agent run Git commands, also check `git log`, because changes it committed no longer appear in `git status` or `git diff`.

Run the Checks Yourself

Run the checks your project instruction file lists, even when the agent says it already did:

```
npm test
npx tsc --noEmit
npm run lint
```

The agent's report that the tests passed is part of its summary. It may have run only some of the tests, run them before its last edit, or reported a result it never saw.

Keep, Fix, Ask, or Discard

After review you have four choices. Keep a change that is right. Fix a few small problems yourself. If something specific is wrong and you can name it, ask the agent for that one correction, such as "the 409 response uses `message` instead of `error`; match the existing error format," then repeat the checks and review (Section 8 covers corrections that do not converge).

If the change would take longer to repair than to redo, discard it. Because you started clean, that takes two steps. The first restores every tracked file:

```
git restore .
```

If the agent staged any of its changes, use `git restore --staged --worktree .` instead. The second step removes the new files the agent created. `git clean` deletes untracked files permanently, so always preview it first:

```
git clean -nd
git clean -fd
```

The first command only lists what would be removed. Run the second only when that list contains nothing of yours. Neither command touches files your `.gitignore` covers, such as `.env` and `node_modules`. Discarding is a normal outcome, and a better task definition in a fresh session is often the fastest route to working code.

Commit with a Note

When you keep a change, update your attribution log (Section 3) first so the entry goes into the same commit. Then commit with a message that says the agent was involved:

```
git add -A
git commit -m "Add schedule-conflict check to enrollment [agent-assisted]"
```

`git add -A` stages everything, which is safe only because you already read the list from `git status`. The `[agent-assisted]` note lets anyone find agent-written changes in `git log`. In a longer task, commit after each step that passes the checks, so a later step that goes badly can be discarded without losing the earlier ones.

Fresh Sessions and Handoff Notes

Start each new task in a fresh session (Table 3 shows how in each tool), so your project instruction file and your task sit near the beginning of a nearly empty context window. Within a task, end the session when the agent starts repeating itself, retrying an approach that already failed, or ignoring a constraint you gave at the start, which are signs of a long session with too much context (Section 2). Do the same when you stop for the day partway through a task. Commit what works, then start the new session with a short handoff note that says what is done, what decisions you made, and what remains:

```
Continuing the schedule-conflict check for
POST /api/sections/:id/enrollments.

Done and committed:
- meetingTimesOverlap() in src/services/enrollment.ts
- Tests in test/enrollment.test.ts for overlapping times, the same day
  without overlap, back-to-back sections, and different days

Decisions:
- Back-to-back sections (one ends at 10:50, the next starts at 10:50)
  do not conflict
- Online sections with no meeting times never conflict

Remaining:
- Call the check from the enrollment route and return 409 with
  { "error": "Schedule conflict with <section code>" }
- Add a route-level test for the 409 response

npm test, npx tsc --noEmit, and npm run lint all pass at the last commit.
```

The agent can draft the note before you end the old session, but check it, especially the decisions, which were often made in conversation rather than in code. Some tools can summarize a long session and continue from the summary, such as `/compact` in Claude Code and in Codex's terminal version. You do not control what that summary keeps, so a handoff note you have read is more predictable.

Working in Parallel with a Worktree

At the Task Delegation level of fluency, you can write code yourself while the agent works on a different task. In the same folder, your edits would mix with the agent's in `git status`, and `git restore .` would discard your work too. Instead, give the agent its own Git worktree, a second working folder linked to the

same repository with its own branch checked out. From your project folder (here, `scheduler/`), with your own work committed, create one for the agent:

```
git worktree add ../scheduler-agent -b agent/conflict-check
cd ../scheduler-agent
npm install
cp ../scheduler/.env .
```

The new folder has every file Git tracks but nothing Git ignores, which is why you run `npm install` and copy `.env` and any other untracked configuration. Open `scheduler-agent` in a separate editor window, or start your terminal agent inside it, and give the agent its task there while you keep working in `scheduler/`.

Run the rest of the loop in the agent's folder, ending with a commit on its branch. Then merge from your own folder and run the checks again, since the merged code includes your work too:

```
cd ../scheduler
git merge agent/conflict-check
git worktree remove ../scheduler-agent
git branch -d agent/conflict-check
```

The last two commands remove the agent's folder and branch, and `git worktree remove` refuses to remove a folder with uncommitted changes. Each tool can also manage worktrees for you (Table 3). Cursor keeps a limited number (25 by default) and automatically deletes older ones beyond that limit, including worktrees you created with `git worktree add`, so commit work in a worktree promptly.

7. Reviewing Everything

Delegating a task does not transfer responsibility for the result. Reading what the agent wrote is what separates Task Delegation from prompting until the program appears to work (Section 1), and a change you accepted without reading is one you cannot explain to your instructor (Section 3).

An Order for Reading a Change

Reading with specific questions finds more than reading from top to bottom and waiting for something to look wrong. The following order works for most tasks:

1. Check the change against your task definition, one requirement at a time
2. Look for changes you did not ask for
3. Check any names and calls you do not recognize against the library's documentation
4. Look at edge cases and error handling
5. Check that the change is consistent with the rest of the codebase
6. Check the security basics

Requirement by requirement. Find the code that satisfies each line of your task definition. For the schedule-conflict check, confirm that the overlap logic is a named export in `src/services/enrollment.ts`, that the 409 body matches the required format, that back-to-back sections do not conflict, and that all four required test cases exist. This step catches partial completion and silent failures (Section 2). If the task said to ask about sections with no meeting times and the agent never asked, find out what the code does with them.

Changes you did not ask for. Anything that no requirement explains needs a reason. Look for these in particular:

- Edits to files outside the task's scope, such as `src/models/section.ts` or anything in `test/starter/`
- Reformatting or reordering of code the task did not touch
- Renamed functions or changed parameters, return types, or response formats
- Deleted tests, or tests whose assertions were loosened
- `.skip` added to a test (`it.skip`, `describe.skip`)
- New `any` types, `as any` casts, or `// @ts-ignore` and `// eslint-disable` comments

The last three groups need the closest attention, because each makes a check pass without making the code correct. A `// @ts-ignore` comment hides the compiler error on the next line, and an `any` type turns off type checking wherever it reaches. Agents often use these when they cannot get a check to pass, and the summary often does not mention them. Unrequested reformatting hides the real edits among many changed lines, so have it removed.

Names and calls you do not recognize. The compiler catches many invented names, but not a real function used on a wrong assumption. Calling `sort()` with no comparison function on an array of numbers compiles and sorts the numbers as strings, so `[9, 10, 1]` becomes `[1, 10, 9]`. Look up unfamiliar functions, options, and packages in the library's documentation rather than asking the agent.

Edge cases and error handling. Ask what the code does with inputs the happy path never sees: a student with no current enrollments, a section with no meeting times, a section ID that does not exist, the same section requested twice. Then look at every `try` block. An empty `catch {}` around the query that loads a student's current sections turns a database error into an empty list, so the conflict check finds nothing and the enrollment goes through.

Consistency with the codebase. Check that the change uses the project's existing error format, naming, and file layout. If the agent added a function that converts `"10:50"` into minutes when `src/utils/time.ts` already has one, you now have two that can drift apart.

Security basics. Input from requests should be validated before it is used. Passwords should be hashed with a password-hashing library (Section 2's login example). Database queries should use parameters, not SQL built by concatenating user input. No API key or other secret should appear in the source code instead of in `.env`.

Reviewing the Agent's Tests

Read the agent's tests as carefully as the code. For each test, ask whether it would fail if the code were wrong. One quick check is to break the code on purpose. Change the comparison in the overlap check from `<` to `<=`, so that back-to-back sections conflict, and run `npm test`. If no test fails, the back-to-back case is not really tested. Undo the change when you are done.

Compare each expected value with the assignment, not with what the code returns (Section 2's `getPage` example), and look for existing tests whose expected values changed, since an agent sometimes edits a test to match the code. For the most important requirements, write the tests yourself, or put the expected values in the task definition. Adding "a section meeting MWF 10:00–10:50 and one meeting MWF 10:50–11:40 do not conflict" to Section 5's task definition would fix the expected answer before any code exists.

Using the Agent During Review

The agent can help you review, but "Is this correct?" tends to get a yes (Section 2). Ask for specific information instead, such as a line-by-line explanation of a function to compare with the code, or a list of the assumptions the code makes about its inputs, which often points at an edge case. You can also give a fresh session, which has not seen the conversation that produced the code, a narrow question:

```
Here is a task definition and the change made for it (git diff below).
List any requirement in the task definition that the change does not
meet. For each one, give the file and the line. Do not modify any files.
```

Treat each finding as something to check in the code, not as a verdict. The reviewing session can report a problem that is not there and miss one that is.

Automated Checks on Your Repository

If your course repository runs the tests and linter on each push, for example through GitHub Actions, those runs catch the check you forgot to run and the file you forgot to commit. They cannot find a missing requirement or a test that expects the wrong value, so do your reading before you push.

The Review Bottleneck

As you get better at defining tasks, the agent produces more code in less time, and reading and integrating that code becomes the largest part of your work. That is expected at the Task Delegation level of fluency. Keep each change small enough to read carefully in one sitting (Section 5), because a change you skim has not been reviewed. If reviewing a change would take longer than writing the code yourself, write it yourself. When review and coordination between tasks take most of your time even with well-sized tasks, you may be reaching the limits of this way of working (Section 10).

8. Debugging and Knowing When to Start Over

Agent-written code will sometimes fail even after a careful review. When you wrote the code, your mental model of each line guides the search, and you have no such model of code an agent wrote unless you build one. The common trap is to skip that work and paste each new error back into the session, accepting fixes you did not examine while the code often gets worse. That is prompting until the program appears to work (Section 1), applied to debugging.

Read the Error Yourself First

Before you give a stack trace to the agent, read it and form your own hypothesis. Suppose a test of the endpoint that lists a student's enrolled sections fails with this output:

```
TypeError: Cannot read properties of undefined (reading 'map')
    at getStudentSchedule (src/services/schedule.ts:42:31)
```

Find out which file and line the error points to, whether that line is the agent's or yours, and what the message says. If the agent's change is not yet committed, `git diff` shows whether line 42 is part of it. If you committed it as a checkpoint, compare against the last commit you know was good:

```
git log --oneline
git diff <last-good-commit> -- src/services/schedule.ts
```

The message says that something on line 42 was `undefined` when the code called `.map` on it, so a value the code expects to be an array is missing in this case. Forming that hypothesis takes a minute or two, and it lets you judge whatever the agent says next.

Narrow It Down with Your Commits

If the tests passed at your last checkpoint commit, `git diff <last-good-commit>` lists every change since. In a large change, look first at the files the stack trace mentions and at shared code the change touched, since a change to a shared function can break code the agent never opened.

Verify Assumptions Before Retrying

Before asking for another attempt, check three assumptions. First, check that the package the code imports is installed at the version the code expects:

```
npm ls <package-name>
```

Second, check the signature of each function the code calls in the library's documentation or type definitions (your editor's Go to Definition opens them), not in the agent's description. Third, check that the agent solved the problem you described. If your task limited conflicts to the student's enrollments in the current term and the code checks every enrollment the student has ever had, the code may be correct for the problem it solved and still wrong for yours.

Ask the Agent to Explain, Not Only to Fix

If you do not see why the code fails, ask the agent to explain it line by line instead of asking it to fix the error:

```
Explain getStudentSchedule in src/services/schedule.ts line by line.
For each line, say what value you expect each variable to hold when
the student has no enrollments. Do not modify any files.
```

The explanation might show that the code treats the result of the enrollment lookup as an array, while the lookup returns `undefined` when no rows match, which the compiler will not flag if the database library types that result loosely. Once you see the assumption, the fix is usually obvious, and you have built the mental model you were missing. The explanation is still generated output and can describe what the code was meant to do instead of what it does, so compare it with the code.

Ask for a Small Fix with Evidence

When you do ask for a fix, give the agent your hypothesis and the evidence for it, and ask for the smallest change that addresses the cause:

```
The test "returns an empty schedule for a student with no enrollments"
fails with this output:

TypeError: Cannot read properties of undefined (reading 'map')
    at getStudentSchedule (src/services/schedule.ts:42:31)

I think findEnrollments returns undefined, not an empty array, when the
student has no rows. Check that against the code. If it is right, make
the smallest change in getStudentSchedule that handles the no-rows case.
Do not change the test or any other file.
```

The prompt asks the agent to check the hypothesis rather than act on it because of sycophancy (Section 2). Review the fix as carefully as the original change, and watch for fixes that hide the symptom: the weakened or skipped tests, ignored errors, and `// @ts-ignore` comments on Section 7's list, or a special case such as an early return for the one student ID the test uses.

You Are the Retry Limit

Nothing stops an agent from proposing fixes indefinitely, each with a confident explanation, so you decide when to stop. As a rule of thumb, if two or three attempts at the same fix make no progress, or each fix introduces a new problem, the approach is probably wrong at a structural level. Discard the change (Section 6) and rethink the task definition. A task that keeps failing is often too large, or it leaves open a decision the agent keeps getting wrong, such as how to treat online sections with no meeting times. Split it or settle the decision, then start a fresh session with the clearer task.

When to Start Over or Write It Yourself

Three situations call for discarding the agent's approach even when retrying might eventually produce passing tests. The first is code that has become harder to understand than what you would write, such as deeply nested logic or unnecessary layers of abstraction. If you cannot explain it, it fails the self-test from Section 3. Replace it with a simpler approach, written by you or delegated again with a tighter definition.

The second is a bug in how the agent's code interacts with things it could not see: your own code, your `.env` settings, the database on your machine, or the way the grading tests call your endpoints. The information the agent needs is not in its context, so debug these yourself.

The third is the core logic the assignment is meant to teach. If a task drifted into it, or the agent's version keeps failing, write it yourself. You will need to explain this part more than any other.

9. Cost

Every tool in Section 4 limits how much agent work you can do, either by capping your usage or by charging for it.

Two Ways to Pay

The first way is a plan. A subscription, a student plan, or an institutional plan such as BYU's ChatGPT Edu access gives you a usage allowance that runs out and then resets on a schedule the tool sets. Until it runs out, one more task costs nothing extra. The Copilot Student plan works this way too.

The second way is billing by usage: API pricing, credits, or pay-as-you-go charges beyond what a plan includes. It works like a meter. OpenAI's student offer for Codex is a block of credits that are used up as you work and expire 12 months after they are granted. Cursor's Premium plan combines the two, with a monthly fee plus separate billing at API prices for Claude and GPT models.

What Uses Up an Allowance

The largest factor is session length. Each time you send a message, the tool sends the whole conversation so far back to the model: your instructions, every file the agent has read, every command and its output, and every earlier reply. The thirtieth turn carries the twenty-nine before it, so each turn costs more than the one before. Retries and large tasks add to that, because failed attempts, file reads, and test output stay in the context for every later turn. Where a tool lets you choose the model, the more capable models typically cost more per request than the faster, lighter ones. The Copilot Student plan selects the model automatically.

The habits that improve quality also keep usage down: small tasks (Section 5), a fresh session for each task (Section 6), stopping after two or three failed fixes (Section 8), and naming the files the agent should read first.

Practical Habits

Allowances tend to run out at inconvenient times, such as the night before a deadline, so start early enough that reaching a limit costs you a day of waiting rather than a late penalty or the entire assignment. If you use any option that bills by usage, set a spending limit before you start and check the tool's usage page from time to time. Cursor's usage-based billing, for example, offers an optional spending cap. Read your tool's current pricing page before you rely on it for a semester, because several of the offers in Section 4 changed during 2026.

What Not to Economize On

Running the build, the tests, and the linter yourself uses none of your allowance. Skipping review saves nothing either, because the problems it would have caught come back as debugging, usually when you have less time. A cheaper model or a thinner context on a hard task often produces a series of failed attempts that use more of your allowance than giving the task what it needed on the first attempt.

10. When You've Outgrown This Way of Working

Signs That Coordination Has Become the Bottleneck

A feature usually takes several tasks: implement the endpoint, write tests for it, review it, update the documentation. Between tasks you do the connecting work yourself, deciding what comes next, copying one task's result into the next prompt, and restating the details the new session does not have. When that work takes more of your time than the tasks themselves, coordination has become your bottleneck.

Three signs usually show up together. The first is that you spend more time carrying outputs between tasks and restating context than producing anything new. The second is bugs that recur across steps rather than within one. The implementation names a function `meetingTimesOverlap`, and the tests written in the next session call `hasScheduleConflict`, because each session started from a slightly different version of the same context. The third is that your tasks have settled into a sequence you could recite, such as implement, test, review, and document, with the same kinds of prompts every time.

Many developers first save the prompts that worked, one for each kind of step, and paste each task's output into the next prompt. That makes each step faster and the work more consistent, but you are still the one

executing the sequence, and it advances only when you carry it forward.

Signs You Are Not There Yet

If your tasks are still often over- or under-scoped (Section 5), or individual outputs still need extensive review before you trust them, the problem is inside the tasks, not between them. Fix that within this way of working first, since automating the steps between tasks does nothing for tasks that are not yet reliable on their own.

What Comes Next

The next level of fluency, Agentic Workflows in Table 1, defines the sequence once. Separate agents take roles such as implementing, testing, and reviewing, and automation carries the work from one step to the next. The developer stops writing production code and improves the sequence: the instructions each agent receives, what passes between steps, and the checks along the way. When an output is wrong, the fix goes into the sequence that produced it, so the next run does not repeat the mistake.

Many developers find this the hardest step on the ladder. It asks you to give up writing code, the work you are best at, and fixing an output by hand still feels productive even though the same problem comes back on the next run.

Choosing a Level for Each Task

Developers at higher levels of fluency still choose a level task by task and do some work the way this guide describes, such as tracking down an unfamiliar bug. For coursework, Task Delegation fits most assignments, which are typically small enough that coordination stays manageable. Each assignment's AI-use level, not your fluency, also sets what you may do (Section 3). An assignment at AI-use Level 3 (Partner) expects you to design and debug alongside the agent, however much of the work you could automate.

My book *Agentic Programming* (agenticprogrammingbook.com) describes the climb from here through the higher levels of the ladder, including how to build the automation that executes these sequences. The higher levels change some of these habits. At Verified Execution, for example, you verify outcomes instead of reviewing code. For your coursework, though, the habits in this guide are the ones to practice: define bounded tasks, review everything the agent produces before you keep it, and keep learning while the agent writes, so you can explain every line you submit.

Appendix A: Task Prompt Templates

These templates follow the five parts of a task definition from Section 5. Fill in the bracketed parts, delete any line that does not apply, and leave out standing rules already in your project instruction file. The commands assume the TypeScript and Node project from Section 4.

Implement a Function or Endpoint

Use this template when the agent is adding new code:

```
Add [function name, or METHOD /api/path] in [file].

Objective: [what it should do, specifically enough that you could tell
whether it does. For an endpoint, give the request, the success
response, and each error response with its status code and body.]

Read first: [the files, types, and existing functions it depends on]

Constraints:
- [Where the code goes, such as a named export in src/services/<name>.ts]
- Follow the existing naming and error format in [file]
```

```
- Do not change [interfaces, response formats, or files that must stay
  the same]
- Do not modify anything in test/starter/

Done when:
- New tests in [test file] cover [the cases, including edge cases]
- npm test, npx tsc --noEmit, and npm run lint all pass

If [a decision the assignment or your instructor should make], ask me
instead of deciding.
```

Fix a Bug

Use this template after you have read the error yourself and formed a hypothesis (Section 8):

```
Fix this bug: [one sentence describing the wrong behavior].

The failing test or command: [test name or command]

The output:
[paste the error message and stack trace, or the expected and actual
values]

My hypothesis: [what you think causes it, and the evidence for it]

Check my hypothesis against the code before changing anything. If it
is wrong, tell me what you found instead of fixing something else. If
it is right, make the smallest change in [file or function] that
addresses the cause.

Constraints:
- Change only [file or files]
- Do not change any test, including the failing one
- Do not skip tests, add empty catch blocks, or add any, as any, or
  // @ts-ignore

If you think the test itself is wrong, stop and explain why instead of
changing it.

Done when:
- [The failing test] passes
- npm test, npx tsc --noEmit, and npm run lint all pass
```

Refactor Code

Use this template when the code should change and its behavior should not:

```
Refactor [code] in [file or files] to [goal, such as moving the
duplicated validation into one shared function].

Behavior must not change. Every input should produce the same result,
response, and error as before.

Read first: [files]

Constraints:
```

```
- Do not modify any test file
- Keep exported names and signatures the same, except [any you intend
  to change]
- Do not reformat or rename code outside [the code being refactored]

Done when:
- npm test passes with the existing tests unchanged
- npx tsc --noEmit and npm run lint pass

If the refactor seems to require a change in behavior or a change to a
test, stop and explain why instead of making the change.
```

Write Tests for Existing Code

Use this template when the code exists and you want tests that check it against the assignment rather than against itself:

```
Write tests for [function or module] in [source file]. Put them in
[test file].

Read first: [source file], [the part of the assignment specification
that applies], and [an existing test file whose style to match]

Cases to cover, with expected values from the assignment:
- [input or situation]: [expected result]
- [input or situation]: [expected result]
- [edge case, such as an empty list or an unknown ID]: [expected result]

Take every expected value from the cases above or from the assignment
specification, never from what the code currently returns.

Do not change [source file] or any file other than [test file]. If a
test fails, leave it failing and report it to me with the expected and
actual values. Do not change the code or the test to make it pass.

Done when:
- Every case above has a test
- npx tsc --noEmit and npm run lint pass
- You have reported the result of npm test, including any failures
```

Explain Unfamiliar Code

Use this template to build your own understanding of code before you review, debug, or change it:

```
Explain [function or file] line by line. Do not modify any files.

For each part, say what it does, what value each variable should hold
for [a typical input, such as a student with two enrollments], and why
it is written that way when the reason is not obvious.

Then list the assumptions the code makes about its inputs and its
environment, such as [a value that is never empty, or a query that
always returns an array]. Say which of those assumptions the code does
not check.
```

Ask for a Plan First

Use this template when you are not sure how a task should be approached (Section 5):

```
I want to [goal]. Before changing anything, describe how you would
approach it.

You may read [files or folders]. Do not modify any files yet.

In your plan, include the following:
- The approach you intend to take
- Each file you expect to change or create, and what changes in it
- Any requirement you are unsure how to interpret
- Any decision the task leaves open that you would otherwise make
  yourself
```

Read the plan as carefully as you would read code. Then approve it as written, or fold your corrections into the task definition.

Attribution-Log Instruction for AGENTS.md

Add this section to your project instruction file to have the agent draft entries in the attribution log from Section 3 and add the file marker:

```
## AI attribution
After each task that changes files, append an entry to
AI-ATTRIBUTION.md in the repository root, creating the file if it does
not exist. Use this format, with today's date:

    ## YYYY-MM-DD: <each file changed, with (new file) after new ones>
    - Tool: <tool name>
    - Task: <the task I gave you, in one or two sentences>
    - Review: TODO

Leave the Review line as TODO. I fill it in after I review the change.
List every file you changed or created, including tests, and do not
list anything you did not change.

If a TypeScript file you created or changed does not already start
with this line, add it as the first line:

    // AI-assisted: see AI-ATTRIBUTION.md
```

Check every entry before you commit it, and replace the TODO with what you reviewed or changed (Section 6).

Appendix B: Glossary

Agent. An AI coding agent: a tool that uses a language model to carry out multi-step coding work, such as reading files, editing code, and running commands.

AI Fluency Ladder. A framework that describes how much of the development process a developer can reliably delegate to AI, from Smart Autocomplete through Autonomous Execution, with Vibe Coding below the ladder (Table 1). Its levels of fluency are unrelated to the CS department's AI-use levels.

AI-use level. The BYU CS department's rating of how much AI may see and write of your work on an assignment, from Level 0 (No AI) to Level 4 (Full Use) (Table 2).

Attribution log. A file named `AI-ATTRIBUTION.md` in the root of your repository, with one entry per agent task recording the date, the files affected, the tool, the task, and what you reviewed or changed afterward (Section 3).

Bounded task. A piece of implementation work with a clear objective, a defined scope, and a complete result you can review, small enough to read carefully in one sitting and large enough to commit on its own.

Checkpoint. A saved state of your work that you can return to, usually a Git commit made after a step passes its checks. Several tools also save checkpoints of their own (Table 3).

Context window. Everything the model can consider when generating a response: your instructions, the conversation so far, the files the agent has read, and the output of the commands it has run. As a long session fills it, instructions given early tend to lose their effect.

Deterministic. Producing the same output every time for the same input. Compilers, linters, and tests behave this way, and the model behind an agent does not.

Diff. A listing of the lines added, removed, or changed between two versions of your files, shown by `git diff` or by your tool's diff view (Table 3).

Hallucination. Output that sounds plausible but is not correct, such as a call to a function that does not exist. It arrives with the same confidence as correct output.

Handoff note. A short note you give a fresh session when you continue unfinished work, stating what is done, what decisions you made, and what remains (Section 6).

Linter. A tool, such as ESLint, that checks source code against a set of rules and flags likely problems. The example project runs it with `npm run lint`.

Permission mode. A tool setting that controls which actions the agent may take without asking you first, such as editing files or running terminal commands (Table 3).

Project instruction file. A short Markdown file of standing instructions, such as build commands, conventions, and files that are off limits, which the agent loads at the start of every session. This guide uses a single `AGENTS.md` (Section 4).

Session. One continuous conversation with an agent, from your first prompt until you clear it or start a new one.

Sycophancy. A model's tendency to shift its output toward what you appear to believe, such as changing a correct answer when you push back.

Task Delegation. The level of fluency this guide teaches: you write code yourself, delegate bounded tasks to an agent, review everything it produces, and integrate the results.

Type check. Running the TypeScript compiler without producing output (`npx tsc --noEmit`) to find errors such as calls to functions that do not exist and arguments of the wrong type. It does not catch names hidden behind `any` or inside strings.

Vibe Coding. Prompting an agent until the program appears to work, without reading the code it wrote.

Worktree. A second working folder linked to the same Git repository, with its own branch checked out, so that you and an agent can work on different tasks at the same time (Section 6).

Resources

Plans, prices, and tool features change often, so check these pages for current details.

BYU Policies and Guidance

- CS Department AI-use policy: cs.byu.edu/department/ai-policy
- Academic Honesty Policy: policy.byu.edu/view/academic-honesty-policy
- Academic integrity guidance for students: ai.byu.edu/academic-integrity-for-students

- BYU AI resources, including ChatGPT Edu access: ai.byu.edu

GitHub Copilot

- Copilot plans: docs.github.com/en/copilot/get-started/plans
- Setting up Copilot Student: docs.github.com/en/copilot/how-tos/copilot-on-github/set-up-copilot/enable-copilot/set-up-for-students
- GitHub education benefits: github.com/settings/education/benefits
- Instruction files in VS Code: code.visualstudio.com/docs/copilot/customization/custom-instructions
- Permissions and approvals in VS Code: code.visualstudio.com/docs/agents/run/approvals

Cursor

- Pricing: cursor.com/pricing
- Rules and `AGENTS.md`: cursor.com/docs/context/rules
- Run modes: cursor.com/docs/agent/security/run-modes
- Worktrees: cursor.com/docs/configuration/worktrees

Claude Code

- Pricing: claude.com/pricing
- Instruction files (`CLAUDE.md` and `AGENTS.md`): code.claude.com/docs/en/memory
- Permission modes: code.claude.com/docs/en/permission-modes
- Worktrees: code.claude.com/docs/en/worktrees

OpenAI Codex

- Student credit offer: developers.openai.com/community/students
- Pricing: learn.chatgpt.com/docs/pricing
- `AGENTS.md`: learn.chatgpt.com/docs/agent-configuration/agents-md
- Approvals and security: learn.chatgpt.com/docs/agent-approvals-security
- Worktrees: learn.chatgpt.com/docs/environments/git-worktrees

Git

- `git worktree`: git-scm.com/docs/git-worktree

Further Reading

- Jerod W. Wilkerson, *Agentic Programming: From Prompts to Production: A Path to AI Fluency*: agenticprogrammingbook.com

Industry Survey

- Gergely Orosz and Elin Nilsson, "AI Tooling for Software Engineers in 2026," *The Pragmatic Engineer*, March 2026: newsletter.pragmaticengineer.com/p/ai-tooling-2026